

Nome: Guilherme de Moraes China Costa
Data: 01/03/2026
Título: Pipelines de CI/CD e DevOps - TP1

Link implementação:

<https://github.com/guilhermecosta/INFNET/tree/master/cicd/TP1-remake>

Parte 1 — Avaliação Teórica

1) O que realmente nasce quando executamos docker build?

Quando executamos docker build, criamos uma imagem Docker, não um container.

Por que o build não cria um container?

O docker build apenas processa o Dockerfile e cria uma imagem (template imutável). A imagem é armazenada no registro local ou remoto. Um container só é criado quando executamos docker run com base nessa imagem.

O que são camadas (layers)?

Uma imagem Docker é composta por camadas de leitura (read-only). Cada instrução no Dockerfile (FROM, RUN, COPY, etc.) cria uma nova camada. As camadas são armazenadas em cache e compartilhadas entre imagens, o que torna o build rápido e as imagens eficientes em termos de armazenamento.

O que significa "congelar decisões técnicas"?

Cada camada é imutável após ser criada. Isso significa que todas as decisões técnicas (sistema operacional, versões de dependências, configurações) ficam "congeladas" na imagem. Isso garante consistência entre ambientes, mas também significa que atualizar algo requer rebuild da imagem.

2) Diferença entre Imagem, Container, Volume e Rede Docker

Conceito	Descrição
Imagem	Template imutável, apenas leitura, usado para criar containers. Contém sistema de arquivos e dependências.

Container	Instância executável de uma imagem. É mutável (pode escrever/modificar arquivos durante execução). Tem seu próprio sistema de arquivos isolado.
Volume	Mecanismo para persistir dados fora do container. Sobrevivem à destruição do container. Usado para bancos de dados e dados importantes.
Rede Docker	Permite comunicação entre containers. Isolamento de rede entre grupos de containers.

Relacionamento no ciclo de vida:

- Imagem é criada via docker build
 - Container é instanciado via docker run a partir de uma imagem
 - Volumes persistem dados independentemente dos containers
 - Redes permitem que containers comuniquem entre si
-

3) Por que scripts em /docker-entrypoint-initdb.d/ rodam apenas na primeira execução?

O PostgreSQL oficial usa o conceito de volume de dados interno. O diretório /var/lib/postgresql/data é onde o banco armazena seus dados.

O papel do diretório de dados e volumes:

- Na primeira execução, se o diretório de dados (/var/lib/postgresql/data) estiver vazio, o PostgreSQL inicializa um novo banco
 - Scripts em /docker-entrypoint-initdb.d/ são executados apenas durante esta inicialização (quando os dados estão vazios)
 - O container verifica se o diretório já possui dados. Se tiver, pula a inicialização
 - O volume Docker (se montado em /var/lib/postgresql/data) persiste os dados entre execuções
 - Por isso, ao usar um volume nomeado ou bind mount, os scripts só rodam uma vez
-

4) Por que usar localhost falha na configuração da API?

Conceito: Isolamento de rede do Docker

Cada container Docker possui sua própria namespace de rede. Quando a API tenta se conectar ao banco usando localhost (ou 127.0.0.1), ela está tentando se conectar a si mesma, não ao container do banco de dados.

Solução:

- Usar o nome do serviço/container como hostname (ex: db ou postgres)
- Ambos os containers devem estar na mesma rede Docker
- O Docker DNS resolve o nome do serviço para o IP do container correspondente

Exemplo correto: jdbc:postgresql://db:5432/postgres Exemplo incorreto:
jdbc:postgresql://localhost:5432/postgres

Parte 2 — Avaliação Prática

Parte 2.1 - Banco Containerizado

Logs do container do banco de dados:

```
guichina@Guichina ~/infnet/cicd/TP1-fuck [master [?21 ]
➤ docker logs ricknmorty-db
The files belonging to this database system will be owned by user "postgres".
This user must also own the server process.

The database cluster will be initialized with locale "en_US.utf8".
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

fixing permissions on existing directory /var/lib/postgresql/data ... ok
creating subdirectories ... ok
selecting dynamic shared memory implementation ... posix
selecting default "max_connections" ... 100
selecting default "shared_buffers" ... 128MB
selecting default time zone ... UTC
creating configuration files ... ok
running bootstrap script ... ok
sh: locales: not found
2026-03-06 12:30:50.682 UTC [35] WARNING: no usable system locales were found
performing post-bootstrap initialization ... ok
syncing data to disk ... ok

Success. You can now start the database server using:

    pg_ctl -D /var/lib/postgresql/data -l logfile start

initdb: warning: enabling "trust" authentication for local connections
initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host, the next time you run initdb.
waiting for server to start...2026-03-06 12:30:51.698 UTC [41] LOG: starting PostgreSQL 17.8 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
2026-03-06 12:30:51.701 UTC [41] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-03-06 12:30:51.707 UTC [44] LOG: database system was shut down at 2026-03-06 12:30:50 UTC
2026-03-06 12:30:51.713 UTC [41] LOG: database system is ready to accept connections
done
server started
```

Listagem de tabelas

```
guichina@Guichina ~/infnet/cicd/TP1-fuck [master [?21 ]
➤ docker exec -it ricknmorty-db psql -U postgres -c "\dt"
List of relations
Schema | Name      | Type | Owner
-----+-----+-----+-----
public | characters | table | postgres
(1 row)
```

Query com select

```
guichina@Guichina ~/infnet/cicd/TP1-fuck [master [?21 ]
➤ docker exec -it ricknmorty-db psql -U postgres -c "SELECT name, status FROM characters LIMIT 5;"
name      | status
-----+-----
Rick Sanchez | Alive
Morty Smith  | Alive
Summer Smith | Alive
Beth Smith   | Alive
Jerry Smith  | Alive
(5 rows)
```

Log dos dois containers funcionando

